

Maximum Points You Can Obtain from Cards

Pattern: Fixed Window Optimization & Sliding Window Complement DE Shaw

Authored by: Arpit Raj
LNMIIT Jaipur
Date: June 30, 2026

Flipkart

Problem Rules & Objective

- You can take a card from either the **start** or the **end** of the array.
- You must pick exactly **k** cards in total.
- **Goal:** Maximize the total score of the picked cards.

Example:

cardPoints = [1, 2, 3, 4, 5, 6, 1], k = 3

Analysis: Total elements **n = 7**. Leaving behind **n - k = 4** contiguous elements.

The window [1, 2, 3, 4] has a sum of 10 (minimum possible). Remaining cards are [5, 6, 1].

Max Score: Total Sum (22) - Min Window Sum (10) = **12**.

Approach 1: Fixed Window Optimization (Sliding Window Complement)

Biggest Observation:

Since you pick exactly **k** cards from the two ends (start/end) combined, you always leave behind exactly **n - k** cards. These left-behind cards will **always form a contiguous subarray**. Therefore, to maximize the score of the **k** picked cards, we simply need to **minimize the sum of the remaining n - k contiguous cards**.

Formula:

$WindowSize = n - k$

$MaxScore = TotalSum - MinWindowSum$

TIME COMPLEXITY	SPACE COMPLEXITY
O(n)	O(1)

C++ Implementation - Sliding Window Complement

```
class Solution {
public:
    int maxScore(vector& cardPoints, int k) {
        int n=cardPoints.size();
        int kk=n-k; //window size

        int total=accumulate(cardPoints.begin(),cardPoints.end(),0);

        if(k==n) return total;
        int currsum=0;
        for(int i=0;i
```

Approach 2: Direct Left/Right Pointer Simulation

Instead of looking at the complement, this approach directly tracks the score of the picked cards by dynamically sliding the picking window from the left side to the right side.

- **Step 1:** Take the first k cards completely from the left side and compute their initial sum.
- **Step 2:** Move one card at a time from the left side to the right side to check every possible combination of elements from the start and end.
- **Step 3:** Track and update the maximum score encountered during this transition.

TIME COMPLEXITY

$O(k)$

SPACE COMPLEXITY

$O(1)$

C++ Implementation - Left/Right Pointer Simulation

```
class Solution {
public:
    int maxScore(vector& arr, int k) {
        int n=arr.size();
        int maxu=0;
        int currsum=0;

        for(int i=0;i=0){
            currsum-=arr[l];
            currsum+=arr[r];
            maxu=max(maxu,currsum);
            r--;
            l--;
        }

        return maxu;
    }
};
```